LIBRARY MANAGEMENT SYSTEM

Full-Stack Enterprise Architecture, Cloud Modernization & Interview Handbook

Live Deployment: `https://librarymanagementsystemproject-l8aw.onrender.com`

Database: TiDB Cloud Serverless (AWS Tokyo) | **Email Gateway:** Brevo v3 HTTPS REST API | **Container:** Docker (Tomcat 9 / JRE 11)

Generated for: Ayush Chowdhury | November 2026

1. Executive Summary & Live Cloud Architecture

The Library Management System (LMS) is an enterprise Java web application engineered to automate academic library operations: member registration, role-based authentication, real-time book catalog search, dynamic book issuing, automatic inventory decrement/increment, iText PDF receipt generation, and two-factor email OTP password recovery.

Originally authored as a legacy NetBeans/Ant project bound to a desktop MySQL server, the application underwent a complete cloud transformation. It was containerized with a multi-stage Docker build, modernized with Maven, and deployed onto Render with TiDB Cloud Serverless MySQL and Brevo Transactional Email REST API.

High-Level Cloud Topology

Tier / Component	Technology / Provider	Operational Role & Configuration
Frontend Client	Bootstrap 5 Responsive UI	Mobile-first, dynamic grid cards, live JS filtering, modal alerts
Application Tier	Apache Tomcat 9 / JRE 11	Render Web Service, multi-stage Docker container, dynamic $PORT
Database Tier	TiDB Cloud Serverless (MySQL 8)	Cloud managed relational DB, SSL/TLS, auto-scaling, persistent tables
Email Delivery	Brevo v3 HTTPS REST API	Bypasses cloud SMTP port 587 block, delivers OTPs & PDF receipts
Document Engine	iText PDF 5.5.13	Dynamic transaction receipt generator embedded directly in servlets
Build & Packaging	Apache Maven 3.9	Standardized pom.xml, automated WAR packaging, CI/CD ready

2. Comprehensive Changelog: Before vs After Modernization

The codebase was comprehensively refactored from a rigid legacy local desktop app to a resilient, production-ready cloud service. The table below documents every key architectural and functional change made:

Area	Legacy State (Before)	Modernized State (After)
Database Connectivity	Hardcoded DriverManager.getConnection("jdbc:mysql://localhost:3306/library","root","...") scattered inside every servlet.	Centralized DBConnection.java. Auto-parses DATABASE_URL, JDBC_DATABASE_URL, DB_HOST/PORT, and provider aliases (TiDB, Aiven, Railway). Automatically enables SSL with trustServerCertificate=true.

Build Automation	Apache Ant build.xml tied to NetBeans IDE paths. Could not build on CI/CD or Docker without NetBeans runtime.	Created clean Maven pom.xml with standard dependencies (mysql-connector-j, itextpdf, javax.mail, cos). Standard 'mvn clean package' builds 44MB self-contained WAR in seconds.
Containerization	No Dockerfile or Compose file. Manual deployment on desktop Tomcat only.	Multi-stage Dockerfile: Stage 1 builds WAR using Eclipse Temurin 11 + Maven; Stage 2 deploys to tomcat:9.0-jre11-temurin with dynamic $PORT binding for Render and memory limits (-Xmx384m).
Mobile UI / Responsive Design	Rigid desktop-only CSS grids (grid-template-columns: 200px 200px 200px 200px), fixed pixel tables, cut off on smartphones.	Upgraded all 12 JSP/HTML pages to responsive Bootstrap 5: mobile drawer toggler, responsive card grids (col-12 col-sm-6 col-lg-3), touch-friendly inputs, overflow-x auto tables.
Inventory & Stock Consistency	Race conditions during book issue/return. Returning a book did not verify whether the user actually borrowed it. Case-sensitive title updates.	Enforced atomic count decrement/increment. Return.java validates loan record in 'issue' table before adjusting stock. AddExisting.java uses case-insensitive LOWER(TRIM(title)) matching.
Email & Cloud SMTP Blocks	JavaMail SMTP over port 587. Render Free Tier blocks outbound SMTP ports 25, 465, and 587, causing socket timeout and breaking OTP.	Integrated Brevo v3 Transactional Email REST API over HTTPS (Port 443). Also added on-screen dev OTP fallback and Render server logging for instant zero-config testing.
Error Diagnostics	LoginServlet masked all database exceptions behind generic alert('Server error occurred. Please try again.').	Sanitized explicit error feedback: catches SQLException, reports connection/credential details safely, and advises exact remediation steps.
Memory & Resource Leaks	Tomcat shutdown threw IllegalStateException from AbandonedConnectionCleanupThread and un-deregistered JDBC drivers.	Refresh.java ServletContextListener shuts down AbandonedConnectionCleanupThread and deregisters all JDBC drivers gracefully on contextDestroyed.

3. End-to-End System Mechanisms & Execution Flows

3.1 Authentication & Role-Based Authorization Flow

1. Client submits credentials via Login.jsp to LoginServlet (POST).
2. LoginServlet queries users table using parameterized PreparedStatement (WHERE Name=? AND email=? AND Role=?).
3. On credential match, session is initialized: session.setAttribute("USER", user), session.setAttribute("Role", role), session.setAttribute("ID", id).
4. Role-based dispatch: Admin is redirected to Admin.jsp; Student is redirected to Home.jsp.
5. All protected pages (Home.jsp, Admin.jsp, MyAccount.jsp, Return.jsp) enforce session guards at top: if session.getAttribute("USER") == null, redirect to Login.jsp immediately.

3.2 Two-Factor Password Recovery & OTP Verification Flow

1. User enters registered email on forgotPassword.jsp and submits to GenerateOtp servlet.
2. GenerateOtp creates a 4-digit cryptographically random code (1000-9999), stores it in session, logs it to server stdout, and redirects to OtpVerification.jsp.
3. OtpVerification invokes EmailUtil.sendEmail(to, subject, text):
   - If BREVO_API_KEY is configured, sends HTTPS POST to https://api.brevo.com/v3/smtp/email (Port 443).
   - If SMTP is used locally, sends via smtp.gmail.com.
4. User enters 4-digit OTP on forgotPassword.jsp; VerifyOtp validates session OTP against user input.
5. Upon verification, Step 3 form is revealed. User enters new password, which is updated in TiDB by PasswordReset servlet.

3.3 Book Browsing, Issuing & iText PDF Receipt Flow

1. Student accesses BrowseBooksServlet: servlet executes SELECT * FROM books, caches list in ServletContext, and renders BrowseBooks.jsp.
2. Student clicks 'Issue Book' on any available title (count > 0). Request submits to Issue.jsp.
3. Issue.jsp decrements book inventory (UPDATE books SET count = count - 1 WHERE book_id = ?), generates issue

record with today's date and due date (+20 days), and inserts into 'issue' table.
4. Forward to CreatePdf servlet: CreatePdf uses iText PDF engine to construct a transaction receipt PDF containing student name, title, issue date, and due date in a styled PdfPTable.
5. CreatePdf forwards to SendUserMail.jsp, which dispatches the generated receipt PDF as an email attachment via EmailUtil to student's inbox.

3.4 Book Return & Inventory Restocking Flow

1. Student opens Return.jsp: dropdown dynamically populates ONLY titles the student has currently borrowed (querying issue JOIN books WHERE U_ID = ?).
2. Student selects title and clicks 'Return Book', submitting to Return.java servlet.
3. Return.java validates loan record existence, removes record from 'issue' table, and increments inventory (UPDATE books SET count = count + 1 WHERE book_id = ?).
4. Refreshes ServletContext 'booklist' so catalog displays real-time stock across all active sessions.

4. Codebase Anatomy & Module Deep Dive

Class / File	Type	Core Responsibilities & Implementation
DBConnection.java	Database Utility	Universal connection factory. Reads DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, DATABASE_URL, and cloud aliases (MYSQLHOST, etc.). Configures autoReconnect, SSL, and trustServerCertificate.
EmailUtil.java	Communication Engine	Multi-channel dispatcher: uses Brevo v3 HTTPS REST API (Port 443) or Resend API when cloud keys exist; falls back to JavaMail SMTP. Supports HTML email, base64 attachments, and detailed error logging.
LoginServlet.java	Controller	Validates user credentials against TiDB database. Establishes HTTP sessions, enforces role-based routing (Admin/Student), and returns explicit diagnostic alerts upon failure.
RegisterServlet.java	Controller	Performs duplicate email/name checks. Inserts new student/admin records, retrieves generated keys, sets session state, and forwards to Login.jsp.
AddBooks.java	Admin Controller	Handles multipart form data via com.oreilly.servlet.MultipartRequest for uploading cover PDFs and images. Validates inventory count, inserts into books table, and updates catalog cache.
AddExisting.java	Admin Controller	Performs case-insensitive title lookup (LOWER(TRIM(title)) = LOWER(TRIM(?))) and adds incremental copies to existing inventory.
Return.java	Student Controller	Validates active loan in issue table for logged-in student, removes issue row, increments available book count, and triggers real-time stock refresh.
CreatePdf.java	Report Controller	Employs iText PDF library to dynamically synthesize official borrowing receipts with table layout, timestamps, and return deadlines, saving to /Issue folder.
Refresh.java	WebListener	ServletContextListener. In contextInitialized(), pre-populates application scope with books and user lists. In contextDestroyed(), terminates AbandonedConnectionCleanupThread to prevent memory leaks.
BrowseBooks.jsp	View (Presentation)	Responsive catalog view featuring real-time client-side JavaScript search filter, stock badges (In Stock / Out of Stock), and touch-friendly issue actions.

5. Database Schema & Relational Integrity

The database is hosted on TiDB Cloud Serverless (MySQL 8 dialect) with InnoDB engine for ACID transaction guarantees, foreign key constraints, and row-level locking:

Table	Column	Data Type	Constraints & Description
users	ID	INT	PRIMARY KEY, AUTO_INCREMENT. Unique member ID.
users	Name	VARCHAR(50)	NOT NULL. User display name.
users	password	VARCHAR(50)	NOT NULL. Account credential.
users	Role	VARCHAR(30)	NOT NULL. Role ('Admin' or 'student').
users	email	VARCHAR(50)	Member email used for OTPs and notifications.
books	book_id	INT	PRIMARY KEY, AUTO_INCREMENT. Unique book ID.
books	title	VARCHAR(250)	NOT NULL. Book title.
books	author	VARCHAR(250)	Author name.
books	pdf	VARCHAR(250)	Uploaded PDF/cover filename.
books	count	INT	Current available physical stock on shelf.
issue	Issue_id	INT	PRIMARY KEY, AUTO_INCREMENT. Transaction ID.
issue	U_ID	INT	FOREIGN KEY -> users(ID) ON DELETE CASCADE.
issue	B_ID	INT	FOREIGN KEY -> books(book_id) ON DELETE CASCADE.
issue	Issue_date	DATE	Date on which book was borrowed.
issue	Due_Date	DATE	Return deadline (default: Issue_date + 20 days).

6. Interview Playbook: How to Explain This Project

6.1 The 2-Minute Elevator Pitch

What to say when asked: 'Tell me about this project'

"I developed and deployed a cloud-native Library Management System built with Java EE Servlets, JSP, MySQL, and Docker, currently live on Render at librarymanagementsystemproject-l8aw.onrender.com.

The system handles member registration, role-based access for Admins and Students, inventory management with real-time stock counters, transaction receipt generation using iText PDF, and two-factor password recovery.

A major engineering highlight was modernizing the app from a local desktop prototype to a resilient cloud deployment: I migrated the build from NetBeans Ant to Maven, containerized it with multi-stage Docker builds, connected it to a cloud TiDB Serverless MySQL database with SSL/TLS, and engineered an HTTPS REST email dispatcher with Brevo to bypass cloud SMTP port blocks."

6.2 Top Technical Questions & Model Answers

Q1: Why did you choose Java Servlets and JSP instead of Spring Boot?

Answer: Servlets and JSP provide foundational mastery of the Java Enterprise web specifications, HTTP lifecycle, ServletContext, HttpSession, and request dispatching without framework magic. While Spring Boot abstracts these mechanics behind annotations like @RestController and @Autowired, building this project on native Servlets demonstrated a deep understanding of connection management, thread safety, classloaders, and manual transaction control.

Q2: How did you solve the SMTP port 587 block on Render?

Answer: Cloud platforms like Render block raw outbound SMTP ports (25, 465, 587) on free tiers to prevent spam. Instead of being blocked by network firewalls, I re-engineered EmailUtil to communicate with Brevo's Transactional Email REST API over HTTPS on Port 443. Port 443 is standard web traffic and never blocked. I also added a fallback mechanism that prints OTPs to server stdout so development and testing can proceed seamlessly.

Q3: How do you prevent race conditions when two users issue the last copy of a book?

Answer: At the database level, inventory count updates are executed using atomic SQL statements ('UPDATE books SET count = count - 1 WHERE book_id = ? AND count > 0'). If the count has already reached 0, executeUpdate() returns 0 rows affected, allowing the servlet to rollback or reject the transaction and notify the user that the book is out of stock.

Q4: How did you fix the Tomcat classloader memory leak on shutdown?

Answer: When MySQL Connector/J registers AbandonedConnectionCleanupThread, it remains active even after webapp context stops, causing Catalina classloader warnings. I implemented a ServletContextListener in Refresh.java that intercepts contextDestroyed(), explicitly shuts down AbandonedConnectionCleanupThread, and deregisters all JDBC drivers from DriverManager.

Q5: How does your Docker multi-stage build optimize deployment?

Answer: In Stage 1 (builder), Maven packages the source code and dependencies into a WAR file using eclipse-temurin:11. In Stage 2 (runtime), we discard the Maven SDK and build tools, copying only the resulting exploded webapp into tomcat:9.0-jre11-temurin. This reduces container image size, avoids exposing build tooling in production, and enables fast container boot times.

6.3 Live Interview Demo Walkthrough Checklist

When demonstrating the live project to an interviewer:
1. Open live link: https://librarymanagementsystemproject-l8aw.onrender.com
2. Show responsive design: Press F12, toggle mobile device view (iPhone/Pixel), show navbar drawer and responsive book cards.
3. Log in as Student (Ayush / ayush52141): Browse books, use the live JavaScript filter bar, issue a book, and show the iText PDF receipt generated.
4. Log in as Admin (Ayush Chowdhury / ayush52141): Open Admin portal, restock existing books or add a new book title, and inspect active user list.
5. Demonstrate Password Recovery: Enter email on forgotPassword.jsp, receive OTP via Brevo, verify, and update password live in TiDB database.